

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

SCHOOL OF COMPUTING
SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH ACADEMY
(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401

Exercise No. 12 Implementation of Memory Allocation strategies (First Fit, Best Fit, Worst Fit)

Objectives:

To implement a C program to simulate the concept of memory allocation strategies. Memory allocation strategies play a significant role in deciding how memory blocks are assigned to running processes.

In this exercise three memory allocation strategies are explored

1. **First Fit**
2. **Best Fit**
3. **Worst Fit**

Each of these strategies allocates memory in a different way, aiming to optimize performance, minimize fragmentation, or balance both.

The **objective** of this lab is to:

- Implement and understand the basic working of these three memory allocation strategies.
- Simulate the process of memory allocation and deallocation.
- Observe how each algorithm handles memory fragmentation and efficiency.

Theory or Concept:

Algorithm for Each Strategy:

a. First Fit Algorithm:

1. Iterate through the list of free memory blocks.
2. Find the **first block** that is **large enough** to accommodate the process. 3. Allocate the memory from that block and update the free block list. 4. If no suitable block is found, the allocation fails.

b. Best Fit Algorithm:

1. Iterate through the list of free memory blocks.
2. Find the **smallest block** that can accommodate the process.
3. Allocate the memory from that block and update the free block list. 4. If no suitable block is found, the allocation fails.

c. Worst Fit Algorithm:

1. Iterate through the list of free memory blocks.
2. Find the **largest block** that can accommodate the process.
3. Allocate the memory from that block and update the free block list. 4. If no suitable block is found, the allocation fails

1. Implement the memory allocation strategies in C or C++:

- Use a structure to represent a memory block with size, start address, and allocation status (free or allocated).
- Implement functions for each of the three algorithms: first_fit(), best_fit(), and worst_fit().

2. Simulate memory allocation and deallocation:

- Create a list of processes with varying memory size requests.
- For each process, simulate memory allocation using all three strategies.
- After each process completes, release the memory and simulate memory deallocation.

3. Generate statistics:

- Track the number of allocations and failures.
- Monitor memory fragmentation (external and internal).
- Display memory usage and efficiency after each allocation and deallocation.

4. **Implement the memory allocation strategies** in C or C++:
   - Use a structure to represent a memory block with size, start address, and allocation status (free or allocated).
   - Implement functions for each of the three algorithms: first_fit(), best_fit (), and worst_fit ().
5. **Simulate memory allocation and deallocation**:
   - Create a list of processes with varying memory size requests.
   - For each process, simulate memory allocation using all three strategies. ○ After each process completes, release the memory and simulate memory deallocation.
6. **Generate statistics**:
   - Track the number of allocations and failures.
   - Monitor memory fragmentation (external and internal).
   - Display memory usage and efficiency after each allocation and deallocation.

CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

typedef struct {
    int id;
    int total_size;
    int remaining_size;
    int start_address;
    bool is_allocated;
} MemoryBlock;

typedef struct {
    int id;
    int size;
```

```
    int allocated_block_id;
    int internal_frag;
} Process;

void simulate_memory_strategy(MemoryBlock originalBlocks[], int m, Process
originalProcesses[], int n, const char* method) {
    MemoryBlock blocks[m];
    Process processes[n];

    for (int i = 0; i < m; i++) blocks[i] = originalBlocks[i];
    for (int i = 0; i < n; i++) {
        processes[i] = originalProcesses[i];
        processes[i].allocated_block_id = -1;
        processes[i].internal_frag = 0;
    }

    int successful_allocations = 0;
    int failed_allocations = 0;

    for (int i = 0; i < n; i++) {
        int chosenIdx = -1;
        if (strcmp(method, "First Fit") == 0) {
            for (int j = 0; j < m; j++) {
                if (!blocks[j].is_allocated && blocks[j].remaining_size >= processes[i].size) {
                    chosenIdx = j;
                    break;
                }
            }
        } else if (strcmp(method, "Best Fit") == 0) {
            for (int j = 0; j < m; j++) {
                if (!blocks[j].is_allocated && blocks[j].remaining_size >= processes[i].size) {
                    if (chosenIdx == -1 || blocks[j].remaining_size < blocks[chosenIdx].remaining_size)
{
                        chosenIdx = j;
                    }
                }
            }
        } else if (strcmp(method, "Worst Fit") == 0) {
            for (int j = 0; j < m; j++) {
                if (!blocks[j].is_allocated && blocks[j].remaining_size >= processes[i].size) {
                    if (chosenIdx == -1 || blocks[j].remaining_size > blocks[chosenIdx].remaining_size)
{
                        chosenIdx = j;
                    }
                }
            }
        }
```

```
        if (chosenIdx != -1) {
            processes[i].allocated_block_id = blocks[chosenIdx].id;
            processes[i].internal_frag = blocks[chosenIdx].remaining_size - processes[i].size;
            blocks[chosenIdx].remaining_size = 0;
            blocks[chosenIdx].is_allocated = true;
            successful_allocations++;
        } else {
            failed_allocations++;
        }
    }

    int total_external_frag = 0;
    for (int j = 0; j < m; j++) {
        if (!blocks[j].is_allocated) {
            total_external_frag += blocks[j].remaining_size;
        }
    }

    printf("\n%s STRATEGY ALLOCATION PATTERN\n", method);
    printf("Process No\tProcess Size\tBlock No\tInternal Frag\n");
    int total_internal_frag = 0;
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", processes[i].id, processes[i].size);
        if (processes[i].allocated_block_id != -1) {
            printf("%d\t\t%d\n", processes[i].allocated_block_id, processes[i].internal_frag);
            total_internal_frag += processes[i].internal_frag;
        } else {
            printf("Not Allocated\t0\n");
        }
    }

    printf("Total Successful Allocations : %d\n", successful_allocations);
    printf("Total Allocation Failures : %d\n", failed_allocations);
    printf("Total Internal Fragmentation : %d KB\n", total_internal_frag);
    printf("Total External Fragmentation : %d KB\n", total_external_frag);
}

int main() {
    int m, n;
    int current_address = 1000;

    printf("Enter number of memory blocks: ");
    if (scanf("%d", &m) != 1) return 1;
    MemoryBlock blocks[m];

    for (int i = 0; i < m; i++) {
```

```
        blocks[i].id = i + 1;
        printf("Size of block %d: ", i + 1);
        if (scanf("%d", &blocks[i].total_size) != 1) return 1;
        blocks[i].remaining_size = blocks[i].total_size;
        blocks[i].start_address = current_address;
        blocks[i].is_allocated = false;
        current_address += blocks[i].total_size;
    }

    printf("\nEnter number of processes: ");
    if (scanf("%d", &n) != 1) return 1;
    Process processes[n];

    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;
        printf("Size of process %d: ", i + 1);
        if (scanf("%d", &processes[i].size) != 1) return 1;
    }

    simulate_memory_strategy(blocks, m, processes, n, "First Fit");
    simulate_memory_strategy(blocks, m, processes, n, "Best Fit");
    simulate_memory_strategy(blocks, m, processes, n, "Worst Fit");

    return 0;
}
```

OUTPUT:

```
guest-w8mnlz@cnlab:~$ gedit exp12.c
guest-w8mnlz@cnlab:~$ gcc exp12.c
guest-w8mnlz@cnlab:~$ ./a.out
Enter number of memory blocks: 5
Size of block 1: 100
Size of block 2: 500
Size of block 3: 200
Size of block 4: 300
Size of block 5: 600

Enter number of processes: 4
Size of process 1: 212
Size of process 2: 417
Size of process 3: 112
Size of process 4: 426

=======================================================
           First Fit STRATEGY ALLOCATION PATTERN
=======================================================
Process No      Process Size    Block No        Internal Frag
-------------------------------------------------------
1               212             2               288
2               417             5               183
3               112             3               88
4               426             Not Allocated   0
-------------------------------------------------------
>> STATISTICS SUMMARY:
Total Successful Allocations : 3
Total Allocation Failures    : 1
Total Internal Fragmentation : 559 KB
Total External Fragmentation : 400 KB
=======================================================

=======================================================
           Best Fit STRATEGY ALLOCATION PATTERN
=======================================================
Process No      Process Size    Block No        Internal Frag
-------------------------------------------------------
1               212             4               88
2               417             2               83
3               112             3               88
4               426             5               174
-------------------------------------------------------
>> STATISTICS SUMMARY:
Total Successful Allocations : 4
Total Allocation Failures    : 0
Total Internal Fragmentation : 433 KB
Total External Fragmentation : 100 KB
=======================================================
```

```
=======================================================
           Worst Fit STRATEGY ALLOCATION PATTERN
=======================================================
Process No      Process Size    Block No        Internal Frag
-------------------------------------------------------
1               212             5               388
2               417             2               83
3               112             4               188
4               426             Not Allocated   0
-------------------------------------------------------
>> STATISTICS SUMMARY:
Total Successful Allocations : 3
Total Allocation Failures    : 1
Total Internal Fragmentation : 659 KB
Total External Fragmentation : 300 KB
=======================================================
```

Observation

For the given process request set for memory in terms of size, show the memory allocation pattern with internal and external fragment size for all three memory allocation strategies (First Fir, Best Fit and Worst Fit)

When processes request memory, First Fit allocates the first available block that fits, Best Fit selects the tightest fitting block to minimize wasted space, and Worst Fit chooses the largest available block. This simulation tracks internal fragmentation within assigned blocks and external fragmentation from unallocated blocks to measure memory efficiency.

Memory Allocation Strategy Comparison

Process No. (Size)	First Fit Allocation	Internal Frag. (FF)	Best Fit Allocation	Internal Frag. (BF)	Worst Fit Allocation	Internal Frag. (WF)
P1 (212 KB)	Block 2 (500 KB)	288 KB	Block 4 (300 KB)	88 KB	Block 5 (600 KB)	388 KB
P2 (417 KB)	Block 5 (600 KB)	183 KB	Block 5 (600 KB)	183 KB	Block 2 (500 KB)	83 KB
P3 (112 KB)	Block 3 (200 KB)	88 KB	Block 3 (200 KB)	88 KB	Block 4 (300 KB)	188 KB
P4 (426 KB)	**Not Allocated**	-	**Not Allocated**	-	**Not Allocated**	-
Total Stats	**Failures: 1**	**Ext. Frag: 400 KB**	**Failures: 1**	**Ext. Frag: 600 KB**	**Failures: 1**	**Ext. Frag: 300 KB**

Result:

Best Fit achieved the highest allocation success rate with minimal internal fragmentation, while Worst Fit left behind the largest internal fragments but preserved larger contiguous blocks for later processes.